

Big Data Capstone Project

Jaulie Goe, Rebecca Glick, Osheen Kaul

May 2025

GitHub repository:

<https://github.com/nyu-big-data/capstone-bdcs-3>

1 Customer segmentation

1.1 Part 1: “Movie Twins”

Link to top 100 most similar user pairs. (Or see top_pairs.txt)

Dataset preparation: In order to filter the initial dataset, we made a few strong assumptions about user similarity. First, we removed all movies that had been rated less than 106 times. This assumes that the most similar users watch mostly popular movies. Of course, there could have been very similar users that were interested in the same niche, unpopular movies, but this assumption was necessary to pare down the dataset to a manageable size. This number was chosen because the bottom 2.2% of movies were watched less than 106 times. It is possible that due to filtering out rarely watched movies, some users’ similarities became artificially inflated (e.g. someone who watches many niche movies and a few popular ones would match with anyone who watched those same popular movies). To handle this, we also filtered by user view count.

The second filter removed users who had seen less than 31 movies. This number was chosen based on what resulted in informative output and the fact that 31 is the mean number of movies watched by all users. Users who had watched very few, popular movies often had seen the same movies as power users, resulting in a similarity that is high but ultimately uninformative.

We ran the experiment multiple times to determine which user view count resulted in the most informative results. Setting the value to less than 10 resulted in the majority of the top 100 users have a similarity of 1. We then set the value to 25 and finally to the mean view count of 31. Between a view count of 25 and 31, the average Spearman correlation between ratings went up from 0.410 to 0.706, so we settled on the latter for our final result.

Our filters assume that the most similar users are users who watch more than the average amount of movies and do not watch a lot of extremely niche movies that are outside of two standard deviations from the mean in terms of number of ratings. For a large dataset, these assumptions are reasonable.

1.2 Part 2: “Movie Twins” Validation

Random users were chosen after the filters were applied to ensure that we are comparing users from the same population. On both the small and large datasets, the correlation was much higher between similar users than between users that were randomly chosen. This is likely because users that watch the same movies are more likely to rate those movies in a similar manner. In other words, users with similar taste in movies exhibit similar judgment. This is also a product of some of the assumptions we made about similarity in Part 1.

Average Spearman correlation for top 100 similar pairs: 0.706

Average Spearman correlation for random 100 pairs: 0.142

2 Movie recommendation

2.1 Part 3: Data Partitioning and Preprocessing

We partitioned the MovieLens ratings dataset by user to ensure that each user appears in only one split—train, validation, or test—thereby preventing data leakage. Using a fixed random seed (e.g., 8372) with Spark’s `randomSplit([0.8, 0.1, 0.1])`, we assigned 80% of users to the training set and 10% each to validation and test sets. Ratings were then joined with these user splits using broadcast joins for efficiency. Initially, we encountered a cold start problem in the latent factor model: users in the validation and test sets had no representation in the training data, making it impossible to generate recommendations for them. To address this, we sampled a small portion (e.g., 20%) of each validation and test user’s ratings and added them to the training set. This ensured that every user was seen during training, enabling the model to produce predictions during evaluation. The resulting splits were saved in parallel as Parquet files with controlled partition counts (e.g., 10 for training, 2 each for validation and test) to optimize write performance. This setup provides a reproducible, scalable, and evaluation-consistent foundation for all subsequent modeling.

For both recommendation systems, we converted the `ratings.csv` file to Parquet to improve Spark read/write efficiency and reduce I/O overhead. The same partitioning logic was applied in both cases to ensure consistency. For evaluation, ground truth labels were defined as the set of movies each user rated above their own average rating, allowing for personalized relevance thresholds. Predictions and ground truth were paired per user to compute standard ranking metrics.

2.2 Part 4: Implementation of Popularity-Based Model

We began our evaluation with a simple popularity-based recommendation model that ranked movies by their average rating. Only movies with at least 20 ratings were retained to ensure that recommendations were based on items with sufficient interaction history. This reduced the impact of noise from rarely rated items, which can artificially inflate popularity scores. This threshold ensures that only movies with sufficient user feedback are considered, improving the reliability and stability of the recommendations. Initially, we defined a “positive” recommendation as any movie a user rated 4.0 or higher. While this provided a stable baseline, we introduced two enhancements. First, we incorporated a Bayesian dampening factor (β) to smooth popularity scores, reducing the influence of movies with very few ratings. Second, we refined the relevance definition to be user-specific: a movie was considered relevant only if a user rated it above their own average. These changes helped the model account for rating sparsity and individual user preferences. After sweeping different β values on a smaller dataset, we observed consistent improvement in evaluation metrics as β increased (Figure 1). We selected $\beta = 50$ for the full dataset because it provided a strong balance, sufficiently smoothing the influence of low-count movies while still allowing highly rated, widely watched titles to retain their ranking advantage on the full dataset.

On the full dataset, evaluation metrics were as follows:

2.2.1 Evaluation

Validation:

- Precision@100: 0.0642
- Recall@100: 0.2063
- NDCG@100: 0.150
- MAP: 0.0392

Test:

- Precision@100: 0.0639
- Recall@100: 0.2044
- NDCG@100: 0.148

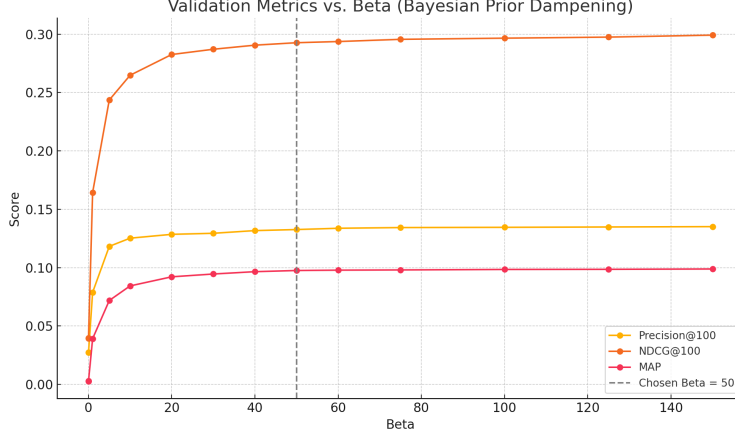


Figure 1: Validation performance of the popularity-based model across different β values. As β increases, smoothing reduces the influence of low-count movies. Precision@100, NDCG@100, and MAP all improve and plateau around $\beta = 150$, but we start seeing diminishing returns around $\beta = 50$.

- MAP: 0.0381

When applied to the full dataset with this configuration, the model achieved a Precision@100 of 0.0642 and an NDCG@100 of 0.150 on the validation set, indicating that relevant items were ranked more effectively than in prior versions. Performance on the test set was comparable, with a Precision@100 of 0.0639 and NDCG@100 of 0.148. The MAP values, 0.0392 (validation) and 0.0381 (test), further confirmed that the model’s ability to rank relevant items was consistent across splits. These results reflect moderate improvements and suggest that incorporating Bayesian dampening and user-specific relevance definitions enhanced the model’s robustness. However, they also reinforce the inherent limitations of popularity-based methods, which rely on global item trends and struggle to capture the nuanced preferences of individual users

2.3 Part 5: Implementation of Latent Factor Model

Our second approach applied Spark’s alternating least squares (ALS) algorithm to learn latent factor representations of users and movies. Once we addressed the cold start issue and had completed the data partitioning and preprocessing, we used the `pyspark.ml.recommendation.ALS` module and tuned two key hyperparameters: the rank and the regularization parameter. After evaluating combinations of `rank` $\in \{50, 100\}$ and `regParam` $\in \{0.01, 0.1\}$ on the validation set in script `als_model.py`, we selected the best-performing model based on mean average precision (MAP). The best hyperparameters were `rank` = 50 and `regParam` = 0.1. To ensure reliable learning, we filtered for users who had rated at least 20 movies in the training set, enabling the ALS model to capture meaningful user profiles.

Following training, we generated top-100 predictions per user and evaluated them using ranking metrics in script `als_test_evaluation.py`. The predicted recommendations were sorted in descending order of predicted score per user before evaluation. We used a personalized definition of relevance, considering movies rated above each user’s mean rating as relevant. For validation, we focused on MAP to guide tuning decisions. For the test set, we computed MAP, NDCG, and Recall at both $K = 5$ and $K = 100$ to understand model behavior under different recommendation depths.

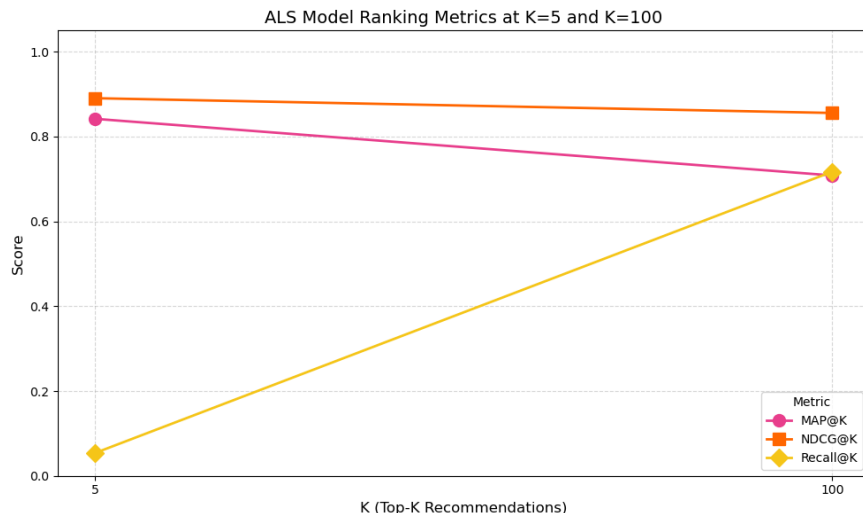


Figure 2: Test set performance of the ALS latent factor model at different values of K . MAP and NDCG are slightly higher for smaller K values, while Recall improves substantially with more recommendations.

2.3.1 Evaluation

Validation (MAP only):

- MAP: 0.5701

Test:

- MAP@5: 0.8419
- NDCG@5: 0.8905
- Recall@5: 0.0539
- MAP@100: 0.7084
- NDCG@100: 0.8557
- Recall@100: 0.7171

These results demonstrate that the ALS model substantially outperforms the popularity-based baseline. It achieves notably higher MAP and NDCG, indicating more effective ranking of relevant items. The strong Recall@100 score shows broad coverage of relevant items among the top predictions. Performance at $k=5$ further highlights the model's precision in surfacing highly relevant items at the very top of the ranked list.

3 Group Contributions

Jaulie focused on identifying movie twins and designing the validation strategy. Rebecca was responsible for writing the data partitioning script to be used for both models and designed the popularity-based model. Osheen handled additional data pre-processing tasks and designed the latent factor model.